Sort and Search

2026-1학기 부경대학교 알고리즘 스터디

목차

- 정렬
  - 정의
  - 버블 정렬
  - 삽입 정렬
  - 병합 정렬
  - C++에서의 정렬
- 탐색
  - 순차 탐색
  - 단조성
  - 이진 탐색
  - lower_bound, upper_bound
  - 매개 변수 탐색
  - 두 포인터

정렬 - 정의

- 정렬(sort)이란?
  - 정렬은 원소들의 집합 혹은 수열을 어떠한 기준에 따라 오름차순 혹은 내림차순으로 재배치하는 것
  - 가령 수열 {1 4 2 10 3}을 오름차순 정렬한다면 수열 {1 2 3 4 10}이 된다.
- 정렬 알고리즘의 성능 평가 기준
  - 시간 복잡도 : 주로 원소 간의 비교 횟수나 이동 횟수를 최선, 최악, 평균의 경우로 나누어 평가함
  - 메모리 : 정렬을 위해 추가적인 배열이나 변수가 얼만큼 필요한지 확인해야 함
  - 안정성 : 우선순위가 동일한 원소들이 입력된 순서로 정렬됨을 보장하는지 확인해야 함

※ 가령 알고리즘이 {c d a_1 a_2 b} 를 사전순으로 오름차순 정렬할 때 {a_1 a_2 b c d} 로 정렬된다면, 안정적인 정렬 알고리즘이라고 할 수 있다.

정렬 – 버블 정렬

- 버블 정렬 (bubble sort)란?
  - 인접한 두 원소를 비교하는 것을 배열의 처음부터 마지막까지 진행하는 과정을 반복해 정렬하는 알고리즘
  - 우선순위가 낮은 원소가 배열의 끝으로 가는 과정이 거품이 수면 위로 올라오는 모습과 닮아 버블 정렬이라고 불림
  - 시간 복잡도가 높으나, 로직과 구현이 간단해 자주 사용됨
  - 안정적인 정렬을 제공함
    - ※ 단, 인접한 두 원소를 비교하므로 인접한 두 원소 a, b의 관계가 a < b일 때만 (동등할 경우 제외) 교환해야 안정적임

정렬 – 버블 정렬

- 코드

```
void bubble_sort(int arr[], int N) {

    // end : 정렬 되지 않은 구간의 마지막 인덱스
    // 즉 처음에는 전 구간이 정렬되지 않은 상태이므로 end = N - 1, 마지막은 앞선 두 원소만 비교하므로 1까지 감소
    for(int end = N - 1; end > 0; end--)
        // cur : 현재 비교하는 두 원소 중 앞선 인덱스
        // 매번 내부 반복문에서 0번째 인덱스에서 end까지 원소가 비교되어야 하므로 end - 1까지 증가
        for(int cur = 0; cur < end; cur++)

            // 만약 왼쪽 원소가 오른쪽보다 크다면 (우선순위가 낮다면) 교환
            if(arr[cur] > arr[cur + 1]) {
                int tmp = arr[cur];
                arr[cur] = arr[cur + 1];
                arr[cur + 1] = tmp;
            }
}
```

정렬 – 버블 정렬

- 버블 정렬 예시
  - 예제) 배열 { 3 5 1 4 2 }를 오름차순 정렬해보자
    - ※ 비교할 두 원소를 각각 빨간색, 정렬이 완료된 원소들을 파란색으로 생각해보자
    - ※ 앞선 코드에서 첫번째 빨간색이 cur, 파란색 직전 원소의 인덱스가 end에 해당한다.
    - ※ 각 줄을 내부 반복문이라고 생각해보자
  - { 3 5 1 4 2 } -> { 3 1 5 4 2 } -> { 3 1 4 5 2 } -> { 3 1 4 5 2 } -> { 3 1 4 2 5 }
    - ※ 가장 우선순위가 낮은 5가 거품처럼 배열의 마지막으로 올라온 것을 확인할 수 있다.
  - { 3 1 4 2 5 } -> { 1 3 4 2 5 } -> { 1 3 4 2 5 } -> { 1 3 2 4 5 }
    - ※ 정렬이 완료된 5를 제외한 4개의 원소에 대해 정렬이 시행된다.
  - { 1 3 2 4 5 } -> { 1 2 3 4 5 } -> { 1 2 3 4 5 }
  - { 1 2 3 4 5 } -> { 1 2 3 4 5 } -> 정렬 끝 !

```
void bubble_sort(int arr[], int N) {
    for(int end = N - 1; end > 0; end--)
        for(int cur = 0; cur < end; cur++)
            if(arr[cur] > arr[cur + 1]) {
                int tmp = arr[cur];
                arr[cur] = arr[cur + 1];
                arr[cur + 1] = tmp;
            }
}
```

정렬 – 버블 정렬

- 버블 정렬의 시간 복잡도
  - 최악 : $O(N^2)$ 비교 연산, $O(N^2)$ 교환 연산
  - 최선 : $O(N)$ 비교 연산, $O(1)$ 교환 연산
    - ※ 적어도 모든 인접한 두 원소 간의 비교를 요구하므로 $O(N)$의 비교 연산이 발생
    - ※ 최선의 경우는 이미 배열이 정렬되어 있는 경우를 의미
  - 평균 : $O(N^2)$ 비교 연산, $O(N^2)$ 교환 연산

정렬 – 삽입 정렬

- 삽입 정렬 (insert sort)이란?
  - 삽입 정렬은 앞부분을 정렬된 상태로 유지하면서, 원소를 알맞은 위치에 삽입해 나가는 정렬 알고리즘
  - 구현이 간단하고, 데이터가 적을 수록 효율적인 알고리즘
  - 원소를 하나씩 제자리에 넣는 방법으로, 직관적인 알고리즘
  - 안정적인 정렬을 제공함

정렬 – 삽입 정렬

- 코드

```
void insert_sort(int arr[], int N) {
    // start : 정렬되지 않은 구간의 첫 인덱스
    //         0번째 원소는 이미 정렬되었다고 가정하므로 1부터 시작
    for(int start = 1; start < N; start++) {
        // target   : 현재 정렬된 구간에 삽입할 원소 값
        // position : target이 최종적으로 삽입될 인덱스
        //            target이 현재 자리 그대로여도 될 수 있으므로 초기값은 start
        int target = arr[start], position = start;

        // 정렬된 구간 arr[0..start-1]을 뒤에서부터 탐색
        for(int cur = start - 1; cur ≥ 0; cur--) {
            // arr[cur] ≤ target 이면 target은 arr[cur] 뒤에 삽입 가능
            if(arr[cur] ≤ target)
                break;

            // target보다 큰 원소는 오른쪽으로 한 칸 이동
            arr[cur + 1] = arr[cur];
            position = cur;
        }

        // 비어진 위치에 target 삽입
        arr[position] = target;
    }
}
```

정렬 – 삽입 정렬

- 삽입 정렬 예시
  - 예제) 배열 { 3 5 1 4 2 }를 오름차순 정렬해보자
    - ※ 정렬이 완료 된 구간을 파란색, 비교 할 두 원소를 빨간색으로 생각해보자.
    - ※ 앞선 코드에서 0 ~ start – 1 까지의 구간이 파란색, 첫번째 빨간색 원소가 cur, 두번째 빨간색 원소가 target이 된다.
    - ※ 각 줄을 내부 반복문이라 생각해보자
  - { 3 5 1 4 2 }
  - { 3 5 1 4 2 } -> { 3 5 1 4 2 }
  - { 3 5 1 4 2 } -> { 3 5 1 4 2 } -> { 1 3 5 4 2 }
  - { 1 3 5 4 2 } -> { 1 3 4 5 2 }
  - { 1 3 4 5 2 } -> { 1 3 4 5 2 } -> { 1 3 4 5 2 } -> { 1 2 3 4 5 } -> 정렬 끝 !

정렬 – 삽입 정렬

- 삽입 정렬의 시간 복잡도
  - 최악 : $O(N^2)$ 비교 연산, $O(N^2)$ 교환 연산
  - 최선 : $O(N^2)$ 비교 연산, $O(1)$ 교환 연산
  - 평균 : $O(N^2)$ 비교 연산, $O(N^2)$ 교환 연산

정렬 – 병합 정렬

- 병합 정렬(Merge Sort)이란?
  - 분할 정복 방식을 사용해 정렬하는 알고리즘
    - ※ 문제를 작은 부분 문제로 나누어 해결하는 알고리즘 방법을 분할 정복 알고리즘이라고 함
  - 배열을 절반 크기로 분할한 뒤, 각 배열을 정렬한 다음 전체 배열을 정렬하는 과정을 재귀적으로 수행
- 동작 원리
    - ※ merge_sort()라는 함수를 생각해보자.
  - 1. 배열의 크기가 2 이상이면 배열을 절반 크기로 2개의 부분 배열로 나눈다. (분할)
    - ※ 만약 크기가 1이라면 이미 정렬된 상태이므로 함수를 종료한다.
  - 2. 각각의 부분 배열에 대해 1번을 적용한다. (즉 merge_sort() 내부에서 merge_sort() 자기 자신을 호출한다.)
  - 3. 두 부분 배열의 원소를 앞에서부터 비교하며 하나의 정렬된 배열로 병합한다.
    - ※ 최종적으로는 전체 배열이 정렬된다.

```c
// arr : 원본 배열
// tmp_arr : 병합 결과를 임시로 저장할 배열
// left, right : 현재 부분 배열의 왼쪽/오른쪽 끝 인덱스
void merge_sort(int arr[], int tmp_arr[], int left, int right) {
    // 부분 배열의 크기가 1 이하이면 이미 정렬된 상태이므로 종료
    if(right - left ≤ 0) return;

    // middle : 현재 부분 배열 [left ~ right]를 반으로 나누는 기준 인덱스
    int middle = (left + right) / 2;

    // 왼쪽 부분 배열 [left ~ middle], 오른쪽 부분 배열 [middle + 1 ~ right]로 분할
    merge_sort(arr, tmp_arr, left, middle);
    merge_sort(arr, tmp_arr, middle + 1, right);

    // 두 정렬된 부분 배열을 병합하여 tmp_arr[left ~ right]에 저장
    int tmp_idx = left, left_idx = left, right_idx = middle + 1;

    while(left_idx ≤ middle && right_idx ≤ right) {
        // 두 부분 배열의 현재 원소를 비교하여 더 작은 값을 tmp_arr에 저장
        // 값이 같다면 왼쪽 원소를 먼저 선택하므로 안정 정렬이 보장된다.
        if(arr[left_idx] ≤ arr[right_idx])
            tmp_arr[tmp_idx++] = arr[left_idx++];
        else
            tmp_arr[tmp_idx++] = arr[right_idx++];
    }

    // 한쪽 부분 배열이 먼저 소진되면, 남아 있는 원소들을 그대로 뒤에 붙인다.
    while(left_idx ≤ middle)
        tmp_arr[tmp_idx++] = arr[left_idx++];
    while(right_idx ≤ right)
        tmp_arr[tmp_idx++] = arr[right_idx++];

    // 병합 결과를 원본 배열의 [left ~ right] 구간에 복사
    for(int i = left; i ≤ right; i++)
        arr[i] = tmp_arr[i];
}
```

정렬 – 병합 정렬

- 병합 정렬 예시
  - 배열 [5, 2, 4, 7, 1, 3, 6, 8] 을 병합 정렬한다고 가정해보자.

[5, 2, 4, 7, 1, 3, 6, 8]

[5, 2, 4, 7] [1, 3, 6, 8]

[5, 2] [4, 7] [1, 3] [6, 8]

[5] [2] [4] [7] [1] [3] [6] [8]

분할

right == left 이므로 더 분할되지 않고 합병된다

[2, 5] [4, 7] [1, 3] [6, 8]

[2, 4, 5, 7] [1, 3, 6, 8]

[1, 2, 3, 4, 5, 6, 7, 8]

합병

정렬 – 병합 정렬

- 병합 정렬의 시간 복잡도
  - 분할 : 원소의 개수가 하나가 될 때까지 반으로 분할하면 총 $\log_2 N$층 이 생김
    - ※ 가령 원소의 개수가 16개면 각 층에서 각 부분 배열의 길이는 16 -> 8 -> 4 -> 2→ 1로, 총4단계를 거쳐 1이 됨
  - 비교 및 삽입 : 각 층에서 모든 원소들이 한번씩 비교 -> O(N)
    - ※ 길이가 같은 두 부분 배열의 앞 원소부터 비교 및 삽입 O(두 배열 길이의 합) -> 모든 부분 배열의 길이의 총합은 N
  - 병합 : 분할의 역순으로 병합되므로 $\log_2 N$번 비교 및 삽입이 발생
  - 따라서 총 시간 복잡도는 O(N logN)
- 병합 정렬의 공간 복잡도
  - 비교 결과를 임시 저장 해야하므로 O(N)

정렬 – C++에서의 정렬

- C++에서의 정렬
  - sort(first, last, pred) : 지정된 범위의 요소를 기준에 따라 정렬하는 함수. <algorithm> 라이브러리에 포함됨
  - first : 정렬할 범위의 첫번째 주소 (iterator, 배열의 시작 주소 등)
  - last : 정렬할 범위의 끝 다음 주소
  - pred : 정렬 함수 (pred(a, b) : a가 b보다 앞서면 true, 아니면 false를 반환하도록 구현해야 함)
    - ※ pred가 없을 경우, 해당 데이터 타입의 오름차순이 정의되어 있을 경우 오름차순 정렬이 디폴트가 됨
    - ※ 이때 두 인수가 동치일 경우 반드시 false 를 리턴 하도록 정렬함수를 짜야 함에 주의하자!
    - ※ 즉 return a <= b와 대신 return a < b와 같이 리턴 해야 한다. (strict weak ordering에 대해 알아보자)
    - ※ 다음 페이지에서 예제를 살펴보자
  - O(N logN)을 보장
    - ※ 빠른 속도 + 메모리 효율성을 위해 인트로 정렬이라는 하이브리드 정렬 (퀵 + 힙 + 삽입 정렬) 사용

```cpp
#include <iostream>
#include <algorithm>
#include <vector>

using namespace std;

// 비교 함수
// a와 b가 순서를 유지한다면 true, 그렇지 않을 경우 false를 리턴하도록 작성
// 10으로 나눈 나머지의 크기가 작은 것이 앞서도록 비교함수를 만들어보자
bool comp(int a, int b) {
    return a % 10 < b % 10;
}

int main() {

    int N = 10;
    int arr[10] = {5, 4, 10, 6, 8, 12, 16, 15, 1, 98};

    // arr[0] ~ arr[N - 1]를 정렬해보자
    // 배열의 이름은 배열의 첫 원소의 주소를 반환하므로
    // 시작 주소는 arr, N번째 주소는 arr + N

    // 오름차순 정렬 (pred 함수 x)
    sort(arr, arr + N);
    for(int i = 0; i < N; i++) {
        cout << arr[i] << " ";
    }
    cout << endl;

    // 10으로 나눈 나머지의 크기 기준으로 오름차순 정렬
    sort(arr, arr + N, comp);
    for(int i = 0; i < N; i++) {
        cout << arr[i] << " ";
    }
    cout << endl;
}
```

```
1 4 5 6 8 10 12 15 16 98
10 1 12 4 5 15 6 16 8 98
```

단어 정렬 BOJ 1181

- 문제
  - 알파벳 소문자로 이루어진 N개의 단어가 입력될 때 아래와 같은 조건으로 정렬하고 순서대로 출력하라.
  - 1. 길이가 짧은 것부터
  - 2. 길이가 같다면 사전 순으로

단어 정렬 BOJ 1181

- 문제
  - 알파벳 소문자로 이루어진 N개의 단어가 입력될 때 아래와 같은 조건으로 정렬하고 순서대로 출력하라.
  - 1. 길이가 짧은 것부터
  - 2. 길이가 같다면 사전 순으로
- 풀이
  - N <= 20,000 이므로 버블 정렬 혹은 삽입 정렬을 사용한다면 시간 초과 ..
  - O(N logN) 시간 복잡도를 가지는 정렬 함수를 사용해 구현하면 정답 !

```cpp
// bj 1181 단어 정렬
#include <iostream>
#include <string>
#include <vector>
#include <algorithm>

using namespace std;

// 비교 함수
bool compare(string a, string b) {
    // 만약 두 문자열의 길이가 같다면, 사전 순으로 정렬하기
    if(a.size() == b.size())
        return a < b;
    // 문자열의 길이가 다르다면 짧은 순으로 정렬하기
    return a.size() < b.size();
}

int main() {
    // 문자열의 개수 N 입력 받기
    int N;
    cin >> N;

    // 문자열들을 관리 할 벡터 정의후 크기 N으로 resize하기
    vector<string> arr(N);

    // N개의 문자열 입력 받기
    for(int i = 0; i < N; i++) {
        cin >> arr[i];
    }

    // 정렬하기
    sort(arr.begin(), arr.end(), compare);

    // 출력하기
    // 중복 출력을 막기 위해 arr[0]를 먼저 출력하고, arr[1]부터는 앞선 문자열과 같은지 확인 후 출력하기
    cout << arr[0] << endl;
    for(int i = 1; i < N; i++) {
        if(arr[i] == arr[i - 1])
            continue;
        cout << arr[i] << endl;
    }
}
```

탐색 – 순차 탐색

- 순차 탐색 (sequential search)란?
  - 배열, 리스트 내부에서 특정한 원소를 찾는 방법
  - 리스트의 첫번째 원소부터 순차적으로 확인하는 탐색 방법
    - ※ 최선의 경우 첫번째 원소가 찾고자 하는 원소인 경우 -> O(1)
    - ※ 최악의 경우 마지막 원소가 찾고자 하는 원소인 경우 -> O(N)
  - 만약 문제가 “x가 배열에 존재하는가” 라는 질문을 M번 한다면?
  - 순차 탐색의 경우 N * M번의 연산이 필요하다.
  - 연산 횟수를 줄일 수 없을까?

탐색 – 단조성

- 단조성 (monotonicity) 이란?
  - 변수가 증가 혹은 감소하는 방향으로만 증감하는 성질을 의미
  - 배열의 경우, 정렬된 경우 단조성을 확인할 수 있음
- 이진 탐색(binary search)란?
  - 정렬된 배열 혹은 리스트에서 찾고자 하는 원소 x의 크기에 따라서 탐색의 범위를 절반으로 줄여가는 탐색 방법
  - “x가 배열에 존재하는가”라는 질문에 앞서, 배열을 오름차순으로 정렬했다고 가정해보자
    - ※ 즉 단조 증가한다고 가정해보자
  - 배열의 중간 값을 봤을 때, x보다 크다면 배열의 오른쪽 부분 또한 x보다 크다!
    - ※ 마찬가지로, x보다 작다면 배열의 왼쪽 부분 또한 x보다 작다고 볼 수 있다.
  - 즉 탐색의 범위가 배열의 왼쪽 절반 / 오른쪽 절반으로 줄어듦을 확인할 수 있다.

```c
// 길이가 N인 정렬된 배열 arr에서 특정한 원소 x가 존재하는지 확인하는 함수
// 만약 존재한다면, 해당 원소의 인덱스를 반환. 없을 경우 -1 반환
int binary_search(int arr[], int N, int x) {

    // left는 현재 탐색 범위의 왼쪽 인덱스
    // right는 현재 탐색 범위의 오른쪽 인덱스
    int left = 0, right = N - 1;

    // 만약 left > right라면 더 이상 탐색 범위를 줄일 수 없으므로 반복문 나오기
    while(left ≤ right) {

        // middle은 탐색 범위의 중간 인덱스
        int middle = (left + right) / 2;
        // 만약 중간 인덱스가 x라면 middle 리턴
        if(arr[middle] == x)
            return middle;

        // 만약 arr[middle]이 x보다 크다면 arr[middle ~ right]는 x보다 크다
        // 따라서 탐색 범위는 left ~ middle - 1로 줄어듦
        if(arr[middle] > x)
            right = middle - 1;
        // 마찬가지로 arr[middle]이 x보다 작다면 탐색 범위는 middle + 1 ~ right가 됨
        else
            left  = middle + 1;
    }
    // 해당 원소가 없으므로 -1리턴
    return -1;
}
```

탐색 – 이진 탐색

- 이진 탐색 예제
  - 배열 [1, 4, 5, 6, 8, 10, 12, 15, 16, 98] 에서 5가 존재하는지 확인해보자
    - ※ 탐색 범위의 첫번째 값을 빨간색, 마지막 값을 파란색으로 칠해보자
    - ※ 즉 앞선 코드에서 빨간색은 left, 파란색은 right 를 의미한다.
  - [1, 4, 5, 6, **8**, 10, 12, 15, 16, 98] ->
    - ※ 중간 값인 8은 찾으려는 값 5보다 크므로 right는 8 직전인 6이 된다.
  - [1, **4**, 5, 6, 8, 10, 12, 15, 16, 98] ->
  - [1, 4, **5**, 6, 8, 10, 12, 15, 16, 98] -> 5가 존재 !
    - ※ c++에서 (2 + 3) / 2 == 2 (나머지는 버림)

탐색 – 이진 탐색

- 이진 탐색의 시간 복잡도
  - 최선의 경우 : O(1)
    - ※ 찾고자 하는 값이 배열의 중간일 경우
  - 최악의 경우 : 배열에 찾고자 하는 값이 없는 경우 or 탐색 범위가 1까지 줄어든 경우
  - 탐색 범위는 절반씩 줄어듦 -> 배열의 길이가 $N / 2^x = 1$을 만족하는 $x = \log_2 N$이 탐색 횟수가 됨
  - 따라서 시간 복잡도는 O(logN)
  - 가령 길이가 500,000 인 정렬된 배열에서 x를 찾는다고 가정해보자
  - 순차 탐색의 경우, 최악의 경우 500000번의 비교 연산이 필요하다
  - 이진 탐색의 경우 $\log_2 500000$, 약 19번의 비교 연산으로 위 질문에 답할 수 있다

숫자 카드 BOJ 10815

- 문제
  - 정수가 적힌 숫자 카드 N개가 주어진다. 그 다음 M개의 정수가 주어진다. M개의 각 정수에 대하여 해당 정수가 적인 숫자 카드가 있다면 1, 없다면 0을 출력하라.

숫자 카드 BOJ 10815

- 문제
  - 정수가 적힌 숫자 카드 N개가 주어진다. 그 다음 M개의 정수가 주어진다. M개의 각 정수에 대하여 해당 정수가 적인 숫자 카드가 있다면 1, 없다면 0을 출력하라.

- 풀이
  - 순차 탐색을 해볼까? -> O(NM)인데, N, M <= 500,000이므로 시간 초과 발생
  - 정렬 후 이진 탐색을 해볼까? -> O(N logN + M * logN) -> O((N + M)log N) !

```cpp
// bj 10815 숫자카드
#include <iostream>
#include <algorithm>
#include <vector>
using namespace std;

int main() {

    // 빠른 입출력을 위해 cin.tie 끊어주기
    cin.tie(0);
    ios::sync_with_stdio(false);

    int N, M;
    cin >> N;

    vector<int> cards(N);

    for(int i = 0; i < N; i++)
        cin >> cards[i];

    // 이진 탐색을 위해 정렬하기
    sort(cards.begin(), cards.end());

    cin >> M;
    // 총 M번 이진 탐색하기
    while(M--) {

        int x;
        cin >> x;
        // 정수 x가 적힌 카드가 있으면 1, 없으면 0
        // 일단 0으로 초기화한 다음, x가 적힌 카드르 탐색 중 발견하면 1 대입하기
        int ok = 0;

        // 이진 탐색
        int l = 0, r = N - 1;
        while(l <= r) {
            int m = (l + r) / 2;

            // 만약 정수 x가 적힌 카드가 있다면 ok를 1로 만들고 탐색 종료하기
            if(cards[m] == x) {
                ok = 1;
                break;
            }
            if(cards[m] > x)
                r = m - 1;
            else
                l = m + 1;
        }
        // ok 출력하기
        cout << ok << " ";
    }
}
```

탐색 – lower_bound, upper_bound

- “정렬된 배열에 x가 존재하는가?”에서, “정렬된 배열에 x가 등장하는 구간은 어디인가?” 로 질문을 확장해보자.
  - 즉 단순히 x의 존재 유무가 아니라, “정렬된 배열에서 x가 차지하는 영역”라는 관점으로 접근해보자.
    - ※ 가령 배열 arr[6] = [1, 2, 3, 3, 3, 4]에서 3의 범위에 대해 생각해보자
- lower_bound란?
  - 정렬된 배열에서 x 이상이 처음 나오는 위치
  - 즉 x가 존재할 수 있는 범위의 가장 낮은 경계선 (lower boundary)을 의미함
    - ※ 배열 arr[6] = [1, 2, 3, 3, 3, 4] 에서 arr[2]부터 3 이상이므로, 3의 lower_bound는 2임을 알 수 있다.
- upper_bound란?
  - 정렬된 배열에서 x 초과가 처음 나오는 위치
  - 즉 x가 존재할 수 있는 범위의 가장 높은 경계선 (upper boundary) + 1를 제공함
    - ※ 배열 arr[6] = [1, 2, 3, 3, 3, 4] 에서 3의 upper_bound는 5, 즉 arr[5] 부터 3 초과임을 알 수 있다.

탐색 – lower_bound, upper_bound

- 활용
  - 길이가 N인 배열에서 x의 개수를 구하는 문제
    - ※ 순차 탐색으로 찾을 경우 O(N)
  - lower_bound는 x가 처음 나오는 위치, upper_bound는 x를 처음 초과하는 위치를 의미함
  - 따라서 x의 개수는 upper_bound – lower_bound
  - -> 즉 질문을 값의 존재 유무에서 값의 범위와 개수 찾기로 확장 가능!

탐색 – lower_bound, upper_bound

- C++에서의 bound 함수
  - <algorithm> 헤더 파일에 lower_bound()와 upper_bound()함수 제공
  - 두 함수 모두 정렬된 배열 혹은 벡터에서 사용해야 하며, first, last, value 3가지 인수를 받아서 각각 lower/upper bound를 반환
  - first : 탐색 범위의 시작 주소
  - last : 탐색 범위의 끝 다음 주소
  - value : lower/upper bound를 구할 값

```
int main() {

    int N = 10;
    int arr[10] = {1, 4, 5, 6, 8, 8, 8, 15, 16, 98};

    // 5의 upper_bound는 arr[3]이므로, (arr + 3)이 리턴됨
    // 포인터가 리턴되므로 값 출력하기
    cout << *upper_bound(arr, arr + N, 5) << endl;

    // 12의 lower_bound는 arr[7]이므로, arr + 6이 리턴됨
    cout << *lower_bound(arr, arr + N, 12);
}
```

```
6
15%
```

- bound함수는 이진 탐색의 탐색 범위를 수정해 직접 구현 가능

```
// 크기가 N인 정렬된 배열 arr에서 x 이상이 처음 등장하는 인덱스 리턴
int lower_bound(int arr[], int N, int x) {
    int left = 0, right = N;
    while (left < right) {
        int middle = (left + right) / 2;

        // arr[middle]이 x와 같을 때도 right = middle
        // 이는 x 이상의 값이 처음 나타나는 '가장 왼쪽 위치'를 찾기 위함
        if (arr[middle] ≥ x)
            right = middle;
        else
            left = middle + 1;
    }
    return left;
}
```

```
// 크기가 N인 정렬된 배열 arr에서 x 초과가 처음 등장하는 인덱스 리턴
int upper_bound(int arr[], int N, int x) {
    int left = 0, right = N;
    while (left < right) {
        int middle = (left + right) / 2;

        // arr[middle]이 x와 같을 때는 left = middle + 1
        // 이는 x를 '초과'하는 값이 처음 나타나는 위치를 찾기 위함
        if (arr[middle] > x)
            right = middle;
        else
            left = middle + 1;
    }
    return left;
}
```

탐색 – 매개 변수 탐색

- 매개 변수 탐색 (paramatric search)란?
  - 조건을 만족하는 최대값/최소값을 찾을 때, “이 값(parameter)이 조건을 만족하는가?”를 확인해 값의 범위를 절반씩 줄여 나가는 기법
  - 찾고자 하는 값의 범위를 이진 탐색과 같은 원리로 찾는 방법
  - 이진 탐색과 달리, x의 존재 유무가 아닌 값이 x일 때의 수식 혹은 함수 자체가 조건을 성립하는지 묻는 방법
  - 이진 탐색과 마찬가지로 단조성을 가져야 함
    - ※ 즉 값이 X일 때 조건을 만족한다면, X보다 큰 값 (혹은 작은 값)에 대해서도 조건을 만족 해야함
    - ※ 문제를 통해 매개 변수 탐색에 대해 이해해보자

예산 BOJ 2512

- 문제
  - N개의 지방이 예산을 요청한다. 만약 요청한 예상을 다 배정할 만큼 총 예산 M이 충분하다면 그대로 배정한다. 그렇지 않을 경우 상한액을 지정해 상한액 만큼만 배정한다. (상한액보다 낮은 예산은 그냥 배정한다.)
  - 가능한 최대의 금액을 배정하였을 때 배정된 예산 중 최대값을 출력하라

예산 BOJ 2512

- 문제
  - N개의 지방이 예산을 요청한다. 만약 요청한 예상을 다 배정할 만큼 총 예산 M이 충분하다면 그대로 배정한다. 그렇지 않을 경우 상한액을 지정해 상한액 만큼만 배정한다. (상한액보다 낮은 예산은 그냥 배정한다.)
  - 가능한 최대의 금액을 배정하였을 때 배정된 예산 중 최대값을 출력하라
- 풀이
  - 만약 각 지방이 요청한 예산의 총합이 국가 예산보다 낮다면 ? 전부 배정할 수 있으므로 정답은 요청 예산의 최대값
  - 그렇지 않을 경우, 지방 당 최대 상한액만큼 배정해야 하므로, 정답은 상한액이 될 것.
  - 따라서 “상한액이 x일 때, 총 예산을 넘지 않도록 지방에 예산을 배정할 수 있는가?” 라고 물었을 때 x의 최대를 찾는 것이 문제
  - 상한액 x에 대하여 위 질문의 답이 “그렇다” 라면, x보다 작은 금액에 대해서도 위 질문이 성립함 -> 단조성 !

예산 BOJ 2512

- 문제
  - N개의 지방이 예산을 요청한다. 만약 요청한 예상을 다 배정할 만큼 총 예산 M이 충분하다면 그대로 배정한다. 그렇지 않을 경우 상한액을 지정해 상한액 만큼만 배정한다. (상한액보다 낮은 예산은 그냥 배정한다.)
  - 가능한 최대의 금액을 배정하였을 때 배정된 예산 중 최대값을 출력하라
- 풀이
  - 즉 상한액 x에 따라서 단조 증가를 보이므로, x의 범위를 줄여가며 상한액의 최대값을 탐색해보자
  - 그래서 상한액이 x일 때, 총 배정금은 얼마가 될까?
  - 순차 탐색을 통해 각 지방의 금액이 x보다 낮을 경우와 클 경우로 나눈다면 O(N log M)의 시간 복잡도로 풀 수 있다.
    - ※ 누적합 및 lower_bound()를 통해 최적화가 가능하다. 따로 고민해보자.

```cpp
// BJ 2512 예산
#include <iostream>
#include <algorithm>
#include <vector>
using namespace std;

int main() {
    // MAX : 예산 요청 중 가장 큰 금액
    int N, M, MAX = 0, ANS = 0;
    cin >> N;
    vector<int> budget(N);
    for(int i = 0; i < N; i++)  {
        cin >> budget[i];
        // MAX에 가장 큰 예산 요청 저장하기
        MAX = max(budget[i], MAX);
    }

    cin >> M;

    // 상한액의 범위는 0 ~ M
    int left = 0, right = M;
    while(left ≤ right) {
        int middle = (left + right) / 2;
        // max budget : 상한액이 middle일 때 배정할 최대 예산액
        // 즉 상한액 혹은 가장 큰 예산 요청 중 더 작은 값이 max budget
        int sum = 0, max_budget = min(middle, MAX);

        // 상한액이 middle일 때 각 지방에 할당한 금액 총합 구하기
        for(int i = 0; i < N; i++)
            sum += min(budget[i], middle);

        // 만약 총합이 총 예산을 넘지 않는다면, max_budget 정답이 될 수 있으므로 ANS에 대입 연산하기
        // 탐색 범위 조정하기
        if(sum ≤ M) {
            ANS = max(ANS, max_budget);
            left = middle + 1;
        } else {
            right = middle - 1;
        }
    }

    cout << ANS;
}
```

탐색 – 두 포인터

- 두 포인터 (two pointers)란?
  - 1차원 배열에서 두개의 포인터를 사용해 특정 조건을 만족하는 구간이나 쌍을 탐색하는 알고리즘
  - 순차 탐색과 달리 구간 연산 혹은 순서쌍과 같이 2개의 포인터가 필요할 때 사용함
  - 배열에서 합이 x인 구간의 경우의 수
  - a[i] + a[j] = k를 만족하는 (i, j)의 개수

수들의 합 2 BOJ 2003

- 문제
  - 길이가 N인 배열 A가 주어진다. A[i] + A[i + 1] + ... + A[j] = M을 만족하는 경우의 수를 구하시오

수들의 합 2 BOJ 2003

- 문제
  - 길이가 N인 배열 A가 주어진다. A[i] + A[i + 1] + ... + A[j] = M을 만족하는 경우의 수를 구하시오
- 풀이
  - 2중 반복문을 통해 i, j를 찾은 다음 구간 합을 구한다면 $O(N^3)$의 시간복잡도를 가진다.
  - 구간의 양 끝점 i, j를 0부터 N - 1까지 한번만 움직일 순 없을까?
  - A[i] + .. + A[j]가 M보다 작다면, j를 오른쪽으로 이동시키며 구간 합이 M보다 같거나 커지는지 확인한다.
  - A[i] + .. + A[j]가 M보다 크다면, i를 오른쪽으로 이동시키며 구간 합이 M보다 같거나 작아지는지 확인한다.
    - ※ 이때 배열의 원소가 30000을 넘지 않는 자연수이므로, 구간의 크기를 늘리고 줄이면 구간 합도 늘어났다가 줄어들게 된다. 즉 구간 합의 증감 방향이 단조적이므로 가능한 풀이!
  - 모든 경우의 수를 확인할 필요가 없으므로 연산 횟수가 줄어든다.
  - 두 포인터 모두 0부터 N – 1까지 움직이므로 총 2 * N번의 덧셈과 뺄셈 연산이 발생 -> O(N)의 시간 복잡도를 가진다.

```cpp
// bj 2003 수들의 합
#include <iostream>
#include <vector>
using namespace std;

int main() {

    // ANS : 경우의 수
    int N, M, ANS = 0;
    vector<int> A(N);

    cin >> N >> M;
    for(int i = 0; i < N; i++)
        cin >> A[i];

    // 두 포인터 i, j 정의
    // 둘 다 0에서 시작하므로, 0으로 초기화
    // 마찬가지로 구간 합 또한 A[0]
    int i = 0, j = 0;
    int sum = A[0];

    while(i < N && j < N) {
        // 만약 오른쪽 포인터가 마지막일 경우, 구간 합이 M보다 작다면 탐색 x
        if(j == N - 1 && sum < M)
            break;
        // 구간 합이 M보다 작을 경우, 오른쪽 포인트 한칸 이동하기 (구간 키우기)
        if(sum < M) {
            j += 1;
            sum += A[j];
        }
        // 구간 합이 M보다 클 경우, 왼쪽 포인트 한칸 이동하기 (구간 줄이기)
        else if(sum > M) {
            sum -= A[i];
            i += 1;
        }
        // 구간 합이 M이라면, 두 포인트 모두 한칸 이동하기 (더이상 한 포인터를 고정하는게 의미가 없으므로)
        else if(sum == M) {
            ANS++;
            sum -= A[i];
            i += 1;

            j += 1;
            sum += A[j];
        }
    }
    cout << ANS;
}
```